MBA ZG583 · MANAGEMENT OF AI PRODUCTS

Experiential Learning Assignment — Capstone Submission

UniConnect Operations OS

An AI-Assisted Academic Operations Platform for a 40-Campus Higher-Education Network

Submission Detail	Entry
Course	MBA ZG583 — Management of AI Products
Assessment type	Experiential Learning Assignment / Capstone (30% weightage)
Project option chosen	Option C — Improve a Real Workflow
Team name	UniConnect (individual submission)
Submitted by	Manoj Kumar Panchothi
Role on the project	Product owner and sole builder of the platform
Product context	NIAT academic operations network, 40 campuses
Date of submission	11 September 2026
Final recommendation	PILOT the AI Intelligence Layer on 5 campuses; scale the platform, stage the AI

HOW TO READ THIS DOCUMENT

This document is the integrated assignment package for the capstone. It contains the executive summary, product strategy, PM artifact pack, UX and trust design, data and evaluation strategy, business and scaling view, ethics and governance note, prototype description, AI collaboration log and individual contribution disclosure, followed by supporting appendices.

Declaration and Academic Integrity Statement

This submission is based on a real, working system that I designed and built, and that is currently in production use across the campus network described in this report. The product exists; the AI capabilities described as "shipped" are running today; the capabilities described as "designed" or "proposed" are not yet live and are clearly marked as such throughout.

Where operational numbers are measured from the system database, they are marked as measured. Where numbers are estimates taken from operator conversations or reasonable modelling, they are marked as estimates and are listed together in Section 13 (Assumptions and Evidence Quality). No research, data, user quotation or product behaviour has been invented for this submission.

AI tools were used extensively in producing this package — for drafting, structuring, generating alternatives, stress-testing edge cases and supporting prototype code. Their use is disclosed in full in Section 11 (AI Collaboration Log). The product decisions, the risk acceptance, the monetisation view and the final recommendation are my own judgement and were not delegated to an AI tool.

Name: Manoj Kumar Panchothi

Date: 11 September 2026

Contents

Section	What it covers
Declaration and Academic Integrity Statement	What is real, what is estimated, and how AI was used
1. Executive Summary	Problem, user, product concept, why AI, and the recommendation
2. Product Strategy and Opportunity Framing	Vision, jobs to be done, opportunity matrix, build vs buy vs API, first wedge
3. PM Artifact Pack	Problem statement, PRD-lite, prioritisation, MVP, roadmap, metrics
4. UX, Workflow and Trust Design	Reviewer journey, screens, explainability, confidence bands, human in the loop
5. Data, Model and Evaluation Strategy	AI task decomposition, data, ground truth, baselines, metrics, go/no-go thresholds
6. Business, Economics and Scaling View	Value logic, cost drivers, unit economics, scaling risks, operations
7. Ethics, Governance and Risk Note	Risk register, harm scenarios, controls, privacy position, launch blockers
8. Prototype and MVP	What is built, what it demonstrates, what it does not yet prove
9. Examination Classroom Allocation	Optimisation AI in the Examinations module: seating allocation, paper-set adjacency, invigilation
10. Course Case Reference and Lessons Transferred	Swiggy as the primary lens, plus four supporting cases
11. AI Collaboration Log	Tools, prompts, where AI was wrong, and the corrections applied
12. Individual Contribution Disclosure	Ownership of each artifact and how outputs were validated
13. Assumptions and Evidence Quality	Measured versus assumed, and the limitations of this submission
Appendix A	Topic Approval Form
Appendix B	Requirement Coverage

1. Executive Summary

1.1 Problem

A higher-education network of 40 campuses has to run the same operational day, 40 times over, every working day. Fees have to be reconciled, daily campus reports have to be written and reviewed, faculty attendance has to be recorded, tasks have to be chased, budgets have to be approved, exams have to be planned, papers have to be generated, seating and invigilation have to be arranged, and problems have to reach the right person before they become escalations.

Until this platform existed, that work lived in spreadsheets, email threads, WhatsApp messages and disconnected Excel files. Three specific failures repeated every single day. First, information arrived late and in inconsistent formats, so a manager reviewing twelve campuses was reading twelve differently written reports. Second, the important signal was buried — a serious problem and a routine update looked identical in an inbox. Third, nothing was auditable: when a decision was questioned two weeks later, nobody could reconstruct who knew what, and when.

The platform solved the structure problem. What remains is a reading and judgement problem: the volume of daily operational text, numbers and documents now exceeds what a small central team can read carefully. That is the problem this capstone addresses.

1.2 Target User and Customer

The primary users are the people who run the operational day: Business Operations Associates (BOAs) and Programme Management Associates at each campus, who submit and act; Programme Managers and the Chief of Staff (COS), who review, sign off and escalate; and the central academic operations and product team, who see the network view. The customer, in the internal sense, is the operations leadership that funds the platform and carries the cost of delays, errors and escalations.

1.3 Product Concept

UniConnect Operations OS is a single role-scoped web platform that replaced the spreadsheet-and-email operating model with one auditable system: one database, one permission model, one record of what happened. It covers fee collection, the daily operations submission spine, tasks, communications and email campaigns, assessments and exam paper generation, examination classroom allocation, faculty and success-coach tracking, budget approvals, meeting intelligence, the operational event calendar and the academic operations hub.

On top of that platform sits the object of this capstone: the UniConnect Intelligence Layer. It is not a chatbot bolted onto a portal. It is a set of narrow, workflow-embedded AI capabilities that read the operational material nobody has time to read, decide how confident they are, and route each item to the right human with the evidence attached. The first wedge — the part recommended for pilot investment — is Daily Operations Triage: turning forty free-text daily campus reports plus their attached numbers into a ranked, evidence-linked review queue for the people who sign them off.

1.4 Why AI Is the Right Lever

Three properties of this workflow make AI the correct instrument rather than a fashionable one. The input is unstructured and written by forty different people in their own words, which defeats rule-based parsing. The judgement being made — "is this campus report normal, or does it hide something that needs attention today?" — is a repeated classification and ranking decision made hundreds of times a week. And the cost of the current approach is not a missing feature but human reading time, which does not fall as the network grows.

Rules, dashboards and thresholds already exist in the platform and they catch the numeric cases well: a fee number that drops, an attendance figure below target, a report that was not submitted by four o'clock. They cannot catch a sentence like "second-year students say the lab machines were down again since Monday" sitting in paragraph four of an otherwise routine report. That is a language problem, and language problems are where current models genuinely outperform rules.

There is a second kind of AI in this platform, and naming it matters because the course rightly warns against treating AI as a synonym for large language models. Allocating students to examination classrooms is a constrained combinatorial problem — every candidate seated, room capacities respected, and and no two neighbours drawn from the same branch or section — solved with search and repair rather than a language model. It is described in Section 9. Language AI reads and drafts; optimisation AI searches and places. Both are AI, they fail in completely different ways, and they need different evaluation. Equally important is where AI is not the right lever. The platform deliberately does not use AI for anything that must be exactly correct and legally defensible — fee arithmetic, permission enforcement, audit records and approval states are all deterministic code, and they should stay that way.

1.5 Recommendation

> Recommendation: PILOT. Run the Intelligence Layer on five campuses for one full academic month in assist-only mode, where every AI output is a suggestion attached to a human decision and no AI output is auto-sent or auto-approved. Continue to scale the deterministic platform across all 40 campuses in parallel. Do not extend AI into exam paper approval, student-facing communication or performance evaluation of staff until the pilot thresholds in Section 5.7 are met.

The reasoning behind a pilot rather than an immediate scale is that the binding risk here is not model quality but trust and workflow fit. If reviewers stop reading because the summary looked convincing, the platform has made oversight worse while appearing to make it faster. That risk is only measurable with real reviewers on real reports, which is exactly what a bounded pilot buys.

2. Product Strategy and Opportunity Framing

2.1 Product Vision

> Every campus in the network runs its day on one auditable system, where AI removes the reading, chasing and re-typing, and people are left with the decisions that actually need judgement.

2.2 Jobs To Be Done

Three jobs anchor the product. They are written from the words operators actually use, gathered during campus visits and daily use of the system.

User	Job statement
Programme Manager / Chief of Staff	When I am reviewing daily reports from ten to twelve campuses in the evening, I want to know which three need me tonight, so that I can act before a problem becomes an escalation instead of reading everything and catching it late.
Business Operations Associate (BOA)	When I am closing my campus day at four o'clock, I want to file one report that is accepted the first time, so that I am not re-sending information and answering the same follow-up questions the next morning.
Academic Operations Lead	When I am planning an examination cycle for a campus, I want exam portions, papers, the exam schedule, classroom seating and invigilation to come out of one consistent flow, so that I am not rebuilding seating charts in spreadsheets and discovering conflicts on exam day.

2.3 Opportunity Framing

Why this problem matters

Operational drag in an education network is not a back-office inconvenience. A delayed fee reconciliation affects a student's access to class. An unnoticed complaint about lab downtime becomes a batch-level satisfaction problem. A conflicting exam timetable is visible to every student and every parent on the day it fails. The cost of poor operations is paid in student experience, which is the thing the institution actually sells.

Why current alternatives are weak

- Spreadsheets scale in file count, not in capability. Forty campuses produce forty versions of the truth and no audit trail.
- A conventional ERP or campus-management suite is built for records, not for a daily operating rhythm. Configuring one to match this network's workflow takes quarters, costs licence fees per user, and still cannot read a free-text report.
- Dashboards report what already went wrong. They are excellent for known numeric metrics and useless for the unstructured half of the day.
- Adding headcount to read reports scales cost linearly with campuses, which is precisely the economics the network is trying to escape.

Why AI may help here

The remaining work after the platform is text-heavy, repetitive and judgement-shaped: read, classify, rank, summarise, route. Modern language models are strong at exactly that shape of task and are

available through an API at a per-item cost measured in fractions of a rupee. Crucially, the platform already provides the two things that make AI usable in an operational setting — structured context to ground the model in, and a human approval state machine to catch it when it is wrong.

2.4 AI Opportunity Matrix

Each candidate capability was scored on business value, technical feasibility and the cost of being wrong. Priority is the resulting judgement, not an arithmetic score.

Candidate AI capability	Business value	Feasibility	Cost of error	Priority
Daily operations triage and severity ranking	High	High	Medium (human sign-off)	P1 — MVP wedge
Daily and weekly narrative summaries for leadership	High	High	Low	P1 — shipped
Meeting intelligence (transcript to summary, actions, attendance)	Medium	High	Low	P1 — shipped
Document intake via OCR (fee receipts, forms, letters)	Medium	High	Medium	P2
Natural-language querying of scoped operational data	Medium	Medium	Medium	P2
Exam paper generation assistance and equivalence checking	High	Medium	High	P3 — assist only
Examination classroom allocation and seating generation (constraint solving)	High	Medium	High	P1 — in the Examinations module
Budget proposal completeness and duplicate-spend checking	Medium	Medium	Medium	P2
Predictive fee-default and dropout risk scoring	High	Low	Very high (student impact)	Deferred
Automated staff performance scoring from operational logs	Low	Medium	Very high (fairness)	Rejected

DECISIONS WORTH DEFENDING

Two entries were deliberately pushed down or out. Predictive student-risk scoring is deferred because the training data available today reflects operational recording habits more than student behaviour, and a wrong prediction attaches to a named student. Automated staff scoring is rejected outright: it converts an operational tool into a surveillance instrument, which would destroy the trust the platform depends on for honest daily reporting.

2.5 Build vs Buy vs Partner / API

The decision was taken layer by layer rather than as a single verdict, because the right answer differs at each layer.

Layer	Decision	Reasoning
Model / inference	API (Google Gemini)	Quality is adequate today, cost is per-call, and switching cost is low because prompts and evaluation live in our code, not in the vendor. Training a model would consume months to reach parity on a task where nothing proprietary is being learned.
Document OCR	API (Google Vision)	A commodity capability. Building it would be an expensive way to reach the same accuracy.
Workflow, permissions, audit, state machines	Build	This is the differentiation. No vendor knows this network's approval ladder, campus scoping rules or daily rhythm, and these are the parts that must be exactly correct.
Database, hosting, queues	Buy (Supabase, Railway, Redis)	Managed infrastructure at a small fraction of the operational cost of self-hosting, with row-level security available at the database layer.
Identity, mail, calendar, storage	Partner (Google Workspace)	The institution already runs on it; integrating is faster and safer than replacing.

Main trade-offs accepted

- Vendor dependency on a single model provider. Mitigated by keeping all prompts, evaluation sets and post-processing inside our codebase and behind one internal interface, so a provider swap is a configuration change and a re-run of the evaluation set.
- Per-call inference cost that grows with usage rather than a fixed licence. Accepted because it keeps the entry cost near zero and the cost is bounded by design (see Section 6.3).
- No proprietary model advantage. Accepted deliberately: the defensibility of this product is workflow depth, permission scoping and audit, not model weights.

2.6 First Wedge

The first wedge is Daily Operations Triage on the existing daily submission spine, restricted to five campuses in one cluster for one academic month. It was chosen because it sits on a workflow that already has a mandatory daily rhythm, an existing human approval step to act as the safety net, complete historical data to evaluate against, and a decision-maker (the reviewing manager) who feels the pain personally every evening. It is narrow enough to be honestly evaluated within a pilot and central enough that success matters.

2.7 Platform or Feature?

This question decides how the work is funded and how it is judged, so it is answered explicitly rather than left implied. UniConnect is a platform: one database, one permission model, one audit trail, with modules added on top of a shared operational spine. The Intelligence Layer, by contrast, is deliberately a feature — a thin capability sitting on that spine, not a separate product with its own data, its own screens or its own login.

That framing carries consequences the team accepts. A feature must earn its place on an existing surface rather than build a new one, which is why triage appears inside the review flow reviewers already use rather than as an "AI assistant" tab. A feature must also be removable: switching the layer off returns the product to the platform the network already runs on, and nothing breaks. If the layer were built as a platform in its own right — its own data store, its own interface, its own notion of a

campus — the network would be running two systems of record, which is the exact problem the platform was built to end.

> Platform where the data, permissions and audit live. Feature where the AI lives. The moment an AI capability starts wanting its own database or its own screen away from the workflow, that is the signal it has been scoped wrongly.

3. PM Artifact Pack

3.1 Problem Statement

> Reviewing managers receive around 40 free-text daily campus reports plus their attached operational numbers every working day. They must decide, within a short evening window, which reports can be signed off and which hide a problem that needs action tonight. Today that decision is made by reading everything at speed, which means the decision quality falls as the network grows, serious issues surface a day or more late, and the reviewer has no consistent record of why one report was escalated and another was not.

3.2 Product Vision for the Intelligence Layer

A reviewer opens one queue instead of forty documents. Each item carries a short factual summary, an explicit severity with the sentence that caused it, a confidence level, and a recommended action. The reviewer agrees, edits or overrides in one click, and every one of those choices becomes both an audit record and a training signal. The reviewer's job changes from reading to deciding — and never from deciding to trusting.

3.3 PRD Summary (PRD-lite): Daily Operations Triage

User problem

The reviewing manager cannot read 40 reports carefully in the evening window available, so severity is discovered late and inconsistently.

Core workflow

1. The Business Operations Associate submits the daily operational report before the 16:00 cut-off, as they do today.
2. On submission, the platform assembles a grounded context package: the report text, the campus's numeric series for the day (fee movement, attendance, open tasks, pending budget approvals), and the last seven days of that campus's history.
3. The model extracts discrete issues from the text, classifies each by type and severity, and produces a two-to-three sentence summary that cites the source sentence for every claim it makes.
4. Deterministic rules run in parallel and can only raise severity, never lower it. A numeric breach always surfaces regardless of what the model concluded.
5. Each report enters the reviewer's queue ranked by severity and confidence, with the evidence attached.
6. The reviewer signs off, sends back, or escalates. Overrides require a one-line reason chosen from a short list plus optional free text.
7. The decision, the model output, the prompt version and the reviewer's reason are written immutably to the audit log.

Where AI creates the value

AI does exactly three things: it reads unstructured text, it proposes a severity with cited evidence, and it drafts a summary. It does not compute numbers, does not decide approvals, does not send anything to anyone, and does not write to any record other than its own suggestion row.

Risks and fallback logic

Failure mode	Fallback behaviour
Model API unavailable or times out	Queue falls back to rules-plus-chronological ordering, with a visible banner saying AI assistance is unavailable. The workflow never blocks on the model.
Model returns low confidence	Item is marked "needs full read", the summary is collapsed by default, and the reviewer sees the raw report first.
Model output fails schema validation	Output is discarded, the item is treated as unprocessed, and the failure is logged to the evaluation table.
Model misses an issue the rules caught	Rule severity wins. The mismatch is flagged into the weekly review sample as a training signal.
Reviewer disagrees with severity	One-click override with reason. Overrides above a weekly threshold trigger a prompt and threshold review.

3.4 Feature Prioritisation

The scoring below is a RICE-style judgement adapted for an AI product. Reach is deliberately constant across these features — every one of them touches the same daily review of the same 40 campuses — so it carries no discriminating information and is left out. Impact and confidence are kept, effort is expressed in developer-weeks for a single developer, and confidence reflects how sure we are that the feature delivers the stated value rather than how sure we are that it can be built. Cost of error, which conventional RICE has no slot for, is carried separately in the opportunity matrix in Section 2.4 and is what moved two high-value ideas out of the plan entirely.

Feature	User value	Effort	Confidence	Priority
Grounded daily report summary with source citation	High	M (2w)	High	P1
Issue extraction and severity classification	High	M (3w)	Medium	P1
Confidence-based queue ranking and routing	High	M (2w)	High	P1
Override capture with structured reasons	High	S (1w)	High	P1
Evaluation harness and weekly sample review	High	M (2w)	High	P1
Rule-and-model reconciliation layer	Medium	S (1w)	High	P1
Cross-campus pattern detection across a week	High	L (4w)	Medium	P2
Natural-language query over scoped operational data	Medium	L (4w)	Low	P2
Document OCR intake for receipts and forms	Medium	M (3w)	Medium	P2

Feature	User value	Effort	Confidence	Priority
Exam paper equivalence assistance	High	L (6w)	Low	P3

3.5 MVP Definition

The MVP is the smallest version that can tell us something we genuinely do not know today. It is five campuses, one cluster, one academic month, assist-only.

In scope: summary generation with citation, issue extraction, severity classification, confidence-based ranking, override capture with reasons, the reconciliation layer with existing numeric rules, the audit trail, and the evaluation harness with a weekly manually reviewed sample.

Out of scope: any automatic approval, any student-facing or parent-facing message, any predictive scoring attached to a named student or staff member, exam paper decisions, and any write access by the model to production operational tables.

The MVP succeeds if: reviewers report that the queue changed the order in which they worked; severity classification reaches the recall threshold on serious issues; the summary factual-error rate stays inside the threshold on the weekly sample; and no serious issue reached leadership later than it would have without the layer.

> The MVP is intentionally narrow because the real risk is trust and workflow fit, not model capability. A wider MVP would produce a more impressive demonstration and a less trustworthy answer.

3.6 Roadmap

Phase	Focus	Proof gate before proceeding
Phase 0 — done	Deterministic platform: submission spine, fees, tasks, approvals, campaigns, exams, audit. Gemini narrative summaries and meeting intelligence live.	Platform adopted as the daily system of record; daily submission compliance stable.
Phase 1 — MVP pilot (1 month, 5 campuses)	Daily operations triage in assist-only mode with full evaluation harness.	Severity recall, summary error rate, override rate and reviewer trust thresholds all met (Section 5.7).
Phase 2 — controlled expansion (1 quarter)	All 40 campuses; cross-campus weekly pattern detection; OCR document intake; budget-proposal completeness checking; scoped natural-language query.	Cost per campus per month within the modelled ceiling; no campus-scoping breach; sustained reviewer agreement rate.
Phase 3 — assisted exam operations	Paper equivalence assistance and portion-completion inference feeding the examinations flow, with mandatory faculty approval.	Zero AI-only paper approvals; faculty acceptance measured; academic governance sign-off obtained.
Phase 4 — evaluate productisation	Assess whether the platform is offered to peer institutions as a product.	Two consecutive quarters of stable internal operation and a defensible support model.

The examination classroom allocation described in Section 9 runs on its own track with its own proof gate, because it is a different kind of AI with a different failure mode. It is not sequenced behind the triage pilot, and neither one blocks the other.

3.7 Success Metrics

Metric type	Metric	Target for pilot
Product	Share of daily reports where the reviewer accepted the AI severity without change	≥ 70%
Product	Median reviewer time spent per campus report	≥ 40% below the pre-pilot baseline
Business	Median time from a serious issue being written down to it being escalated	Same working day, versus a baseline of one to two days
Business	Reports sent back for missing information	≥ 30% reduction
Trust / guardrail	Severity recall on serious (P1) issues in the weekly sample	≥ 0.90
Trust / guardrail	Summary statements not supported by the source report	≤ 2% of sampled statements
Trust / guardrail	Reviewer override rate	Between 10% and 25% — below 10% suggests rubber-stamping, above 25% suggests the model is not useful
Trust / guardrail	Cross-campus data exposure incidents	Zero, non-negotiable

A METRIC THAT CAN FAIL UPWARDS

The override band is deliberately two-sided. A very low override rate is not a success signal; it is the most likely early symptom of automation complacency, which is the specific failure this design is most afraid of.

3.8 How These PM Artifacts Changed Because the Product Is AI-Powered

Standard artifact	What changed in this product
Problem statement	Had to name the repeated decision being improved ("which report needs me tonight"), not the feature being built. The unit of value is a decision, not a screen.
Persona and user story	Every story carries an error cost. The reviewer story explicitly states what happens when the suggestion is wrong, because that path is not an exception here — it is a normal branch.
PRD	Gained three sections a conventional PRD does not have: data assumptions and grounding sources, fallback logic for every failure mode, and prompt or model versioning as a first-class change.
Prioritisation	Cost of error was added as a scoring dimension alongside value and effort. It is what moved predictive student-risk scoring out of the MVP despite high value.

Standard artifact	What changed in this product
Roadmap	Phases end at proof gates rather than at ship dates. Phase 2 cannot start because Phase 1 finished; it starts because Phase 1 met measured thresholds.
Metrics	Model metrics alone are meaningless to operations, so each technical metric is paired with the business outcome it is supposed to move, plus a guardrail metric that can veto a launch.
Launch checklist	Now includes prompt version pinning, rollback to rules-only in a single configuration flag, an evaluation set run before release, and an escalation owner named for the first week.
Pricing / cost note	Cost per unit of work is now variable and usage-linked, so the cost model is built per report and per campus rather than per seat.
Review memo	Separates what the system does today from what a demonstration suggests it could do, because the gap between those two is where AI projects usually lose credibility.

4. UX, Workflow and Trust Design

4.1 Design Principle

> The system behaves like a well-prepared assistant who has read everything and is briefing you, not like a manager who has already decided. Every AI output is visibly a suggestion, is attached to its evidence, and is one click away from being overruled.

4.2 Main Workflow

The journey below is the reviewer's evening, which is the moment the product is designed around.

1. The reviewer opens the Review Queue. Items are grouped into three bands: "needs you tonight", "review normally" and "ready to sign off".
2. Each card shows the campus, a one-line summary, the severity chip, the confidence indicator and the number of extracted issues.
3. Opening a card shows the AI summary on the left and the original report on the right, with the sentences that generated each issue highlighted in the original text.
4. Numeric context sits beneath: today's fee movement, attendance, open tasks and pending budget approvals for that campus, with the seven-day trend.
5. The reviewer chooses sign off, send back with a note, or escalate. Escalation pre-fills a task or communication draft that the reviewer must edit and confirm before it goes anywhere.
6. Every action writes to the audit trail with the model output and prompt version recorded alongside the human decision.

4.3 Key Screens

Screen	What it must make obvious
Review Queue	Which campuses need attention first, and that the ordering came from a suggestion the reviewer can re-sort or ignore.
Report Detail (split view)	That every AI sentence traces to a real sentence in the submitted report. Nothing is asserted without a visible source.
Issue Panel	Issue type, severity, confidence, the cited evidence, and a one-click "not an issue" or "more serious than this" control.
Numeric Context Strip	That numbers come from the deterministic system, not from the model — visually distinct from AI-generated text.
Override Dialogue	That the reason is required, short to give, and will be read by a human during the weekly review.
Weekly Evaluation View (internal)	How the model performed against the sampled ground truth, so the team argues with data rather than impressions.

Screenshots of the live production screens and the pilot wireframes are supplied in the Appendix folder referenced in Section 8.6.

4.4 Trust and Explainability

What the user must be able to see

- The exact source sentence behind every extracted issue.
- A plain-language confidence statement — "high confidence", "worth checking", "read this one fully" — rather than a raw probability, which operators consistently misread.
- A visual distinction between deterministic facts (numbers from the database) and generated text (summaries, severities).
- The fact that the item has not been actioned by anyone else, and who else has viewed it.

What is deliberately hidden

- Raw model scores and internal probabilities, which invite false precision.
- The prompt itself, which is a maintenance artifact rather than user information, though the prompt version identifier is recorded in the audit log and available to the product team.
- Any suggestion at all when confidence is below the floor — a bad suggestion is worse than no suggestion, because it anchors the reviewer.

4.5 Uncertainty Handling

Confidence band	System behaviour
High	Summary shown expanded, severity chip shown, item placed in the recommended band. Sign-off is a single action, and the original report is still one click away.
Medium	Summary shown with a "verify before acting" marker. The recommended action is shown as a question rather than a statement. Item cannot be bulk-signed-off.
Low	No summary shown by default. Item is labelled "needs full read", ordered by rules and recency, and the original report opens first.
Failure or no response	Rules-only mode with a visible banner. The reviewer's workflow is unchanged from the pre-AI experience, which is the definition of a safe failure here.

4.6 Human in the Loop

The human review point is not a new control invented for the AI feature. It is the approval step that already existed in the daily submission state machine, which is why this wedge was chosen first: the safety net was already load-bearing and already trusted.

- No report changes state without a human action. The model can order the queue; it cannot advance the state machine.
- Escalations and outbound messages are drafts until a human edits and confirms them.
- Overrides capture a structured reason, and those reasons are read every week — both as a quality signal and as a check on whether reviewers are engaging or rubber-stamping.
- Auto-sign-off, which exists in the platform for reports left unreviewed past the deadline, is disabled for any report the layer marked as serious. A serious item must be seen by a person.

5. Data, Model and Evaluation Strategy

5.1 The Problem Expressed as AI Tasks

"Triage" is not one machine-learning task. Decomposing it honestly is what makes the evaluation possible, because each component fails differently and needs its own metric.

Component	Task type	Approach
Issue extraction	Information extraction from free text	LLM with a strict output schema and mandatory source-sentence citation
Severity classification	Multi-class classification	LLM with a written rubric, reconciled with deterministic numeric rules
Summarisation	Grounded abstractive summarisation	LLM constrained to the supplied report and numeric context only
Numeric anomaly detection	Time-series outlier detection	Deterministic statistics on the seven-day campus series — no model involved
Queue ranking	Ranking	Deterministic function of severity, confidence, campus history and age
Document intake (Phase 2)	OCR plus field extraction	Vision API followed by schema-validated extraction
Natural-language query (Phase 2)	Text to structured query	Constrained generation over pre-approved, permission-scoped views only

5.2 Data Sources

- Internal, primary: submitted daily reports, task records, fee movement, attendance records, approval history, communication logs and the immutable audit trail — all held in the platform's own event-sourced Postgres database.
- Internal, secondary: fee data synchronised from institutional Google Sheets, and meeting transcripts pulled from Drive for the meeting-intelligence capability.
- User-generated: reviewer overrides and their structured reasons, which are the most valuable dataset the product creates because they are ground truth produced as a by-product of normal work.
- External: none. No third-party data is used, and no operational data leaves the institution's systems other than the API call to the model provider, which is covered in Section 7.

> **ASSUMPTION WORTH STATING OUT LOUD**
>
> Known data-quality risk: the fee figures originate in a spreadsheet maintained by people, and spreadsheet accuracy is the single largest data-quality exposure in the whole platform. Any AI capability that consumes those figures inherits that exposure, which is why the numeric layer stays deterministic and visibly separated from generated text.

5.3 Labelling and Ground Truth

There is no pre-existing labelled dataset for "was this campus report serious?", and inventing one would be the fastest way to build a system that scores well and helps nobody. Ground truth is therefore constructed in three layers.

1. Implicit labels: every reviewer decision — signed off, sent back, escalated — is a label attached to a report, generated for free by work that would happen anyway.
2. Explicit labels: a stratified sample of 30 reports per week is labelled independently by an experienced reviewer using the same written severity rubric given to the model, before seeing the model output. This is the evaluation set of record.
3. Correction labels: every override with its reason, which is where the model's systematic mistakes become visible as patterns rather than anecdotes.

The lesson taken from the iMerit case is that labelling quality, rubric clarity and labeller agreement matter more than labelling volume. Practically, that means the rubric is versioned in the same repository as the prompt, the weekly labeller and the model see the same rubric text, and a disagreement between two human labellers is treated as a rubric defect to be fixed rather than noise to be averaged away.

5.4 Baseline

The model is not compared against nothing, and not against a hypothetical perfect reviewer. It is compared against two concrete alternatives that could be implemented for far less money.

Baseline	What it represents	Why the comparison matters
Chronological queue	Today's behaviour — read reports in the order they arrive	If AI ordering does not beat arrival order, the feature has no reason to exist
Rules-only ordering	Numeric thresholds and keyword flags, buildable in about a week	This is the honest bar. AI must add value beyond what a simple rule set already catches, specifically on issues that are only expressed in prose

The pilot runs the rules-only baseline in parallel on the same reports throughout, so the comparison is measured rather than argued.

5.5 Model and API Choice

Google Gemini is used through its API, for four reasons. It is already integrated into the platform and covered by the institution's existing Google Workspace relationship, which materially simplifies the data-handling conversation. Its quality on grounded summarisation and extraction is sufficient for an assist-only workflow. Its cost per report is small enough that the economics work at 40 campuses. And the work required to move to another provider is limited to one internal interface plus a re-run of the evaluation set.

Fine-tuning was considered and rejected for the MVP. There is not yet enough labelled data to fine-tune responsibly, a fine-tuned model would freeze a rubric that is still being refined, and the current failure modes look like prompt, rubric and grounding problems rather than capability problems. Fine-tuning is revisited only if the evaluation set shows a persistent gap that better prompting and grounding cannot close.

5.6 Evaluation Framework

Metric type	Metric	Why it matters
Technical	Severity classification recall on P1 issues	Missing a serious issue is the failure that actually hurts; recall is prioritised over precision here
Technical	Severity classification precision overall	Too many false alarms and reviewers stop trusting the ordering entirely
Technical	Issue extraction F1 against the weekly labelled sample	Measures whether the layer sees what a careful human reader sees
Technical	Groundedness — share of summary statements traceable to a source sentence	The direct measure of hallucination in this product
Technical	Schema validation failure rate and median latency	Operational reliability; the queue must be ready when the reviewer opens it
Business	Reviewer minutes per campus report	The efficiency claim, measured rather than asserted
Business	Time from issue written to issue escalated	The outcome the institution actually cares about
Business	Send-back rate for incomplete reports	Indicates upstream quality improvement, not just downstream speed
Trust / guardrail	Override rate, within a two-sided band	Detects both an unhelpful model and a disengaged reviewer
Trust / guardrail	Serious issues surfaced only by rules and missed by the model	The clearest early-warning signal of model drift
Trust / guardrail	Campus-scoping violations detected in output	A single breach is a launch blocker, not a metric to optimise

5.7 Launch Thresholds — Go / No-Go

These thresholds were set before the pilot, which is the only time thresholds can be set honestly.

Condition	Threshold	If not met
P1 severity recall on the weekly labelled sample	≥ 0.90 sustained over four weeks	Do not expand. Improve rubric and grounding, re-run.
Ungrounded summary statements	≤ 2% of sampled statements	Tighten grounding constraints; suspend summaries if above 5%.
Reviewer override rate	Between 10% and 25%	Below 10%: investigate complacency directly with reviewers. Above 25%: the model is not adding value yet.
Campus-scoping or permission violations	Exactly zero	Immediate stop. This is an absolute launch blocker.

Condition	Threshold	If not met
Inference cost per campus per month	Within the modelled ceiling in Section 6.4	Reduce context size and batch, or restrict the capability to serious items only.
Reviewer confidence (structured feedback)	Majority state the queue changed their working order for the better	Treat as a workflow-fit failure rather than a model failure, and redesign the surface.

6. Business, Economics and Scaling View

6.1 Monetisation Model

The platform is an internal product, so the primary monetisation model is an internal value case: it must return more in avoided cost and avoided failure than it consumes in infrastructure, inference and engineering time. This is a stricter test than an external product faces, because there is no revenue line to hide behind and no customer willing to pay for a feature that does not work.

A secondary path exists and is deliberately deferred to Phase 4. Peer institutions running multi-campus programmes have the same workflow problem, and the platform is already multi-tenant by construction — the university is the scoping key on almost every table, and permissions are enforced at the database layer. If it were offered externally, the model would be a per-campus annual subscription with tiered inclusion of the Intelligence Layer, plus a usage allowance for inference. That path is not recommended now: a product with one expert operator and a single builder is not yet supportable for external customers, and saying so is more useful than pretending otherwise.

6.2 Value Logic

Value source	How it materialises
Reviewer time recovered	Managers spend evening hours deciding rather than reading, and the hours saved do not scale up with campus count.
Earlier problem detection	Issues surface on the day they are written rather than one to two days later, which converts escalations into routine fixes.
Fewer round trips	Better first-time report quality reduces send-backs and the follow-up messages they generate.
Auditability	Every decision is reconstructable, which reduces the cost of disputes and reviews and is worth real money in a regulated academic setting.
Replaced tool sprawl	One platform instead of many spreadsheets, mail threads and side files, removing coordination cost that never appeared in any budget line.

6.3 Cost Drivers

Cost driver	Behaviour as the network grows
Model inference (Gemini API)	Linear in reports processed and in context size. The largest AI-specific variable cost and the one that needs a ceiling.
OCR (Vision API), Phase 2	Linear in documents processed; bounded by document volume, which is predictable.
Managed infrastructure (database, hosting, Redis, queues)	Steps up in tiers rather than growing smoothly; predictable and small relative to staff cost.
Engineering and maintenance	The dominant true cost. Currently concentrated in one person, which is a risk as much as a cost — see Section 6.6.

Cost driver	Behaviour as the network grows
Human review time	Does not disappear and must not. It is the safety mechanism, and the design intent is to make it shorter, not to remove it.
Evaluation overhead	Roughly two to three hours of experienced-reviewer time each week for the labelled sample. Cheap, and non-negotiable.

6.4 Unit Economics View

The unit is one campus for one month. The test the product must pass is stated as an inequality rather than a forecast, because the inputs on the left are measurable during the pilot and the inputs on the right are already known.

> Monthly inference and infrastructure cost attributable to one campus, plus the amortised engineering cost of maintaining the layer, must remain materially below the loaded cost of the reviewer and associate hours it removes, at every campus count between five and forty.

Three structural properties make that inequality likely to hold, and each is testable during the pilot. First, the input is small: a daily report plus a compact numeric context is a modest amount of text, and roughly twenty-two working days per campus per month bounds the call volume tightly. Second, the expensive human hours being displaced are managerial rather than clerical. Third, infrastructure cost is shared across all 40 campuses rather than duplicated per campus.

Two properties could break it, and both are controlled by design decisions rather than hope. If context size grows — for instance by including thirty days of history instead of seven — cost grows with it, so context size is a deliberately pinned configuration value. And if the layer is extended to high-volume, low-value item types, the call count could rise by an order of magnitude while the value per call falls; this is why expansion is gated on measured value per capability rather than on enthusiasm.

The actual rupee figures are computed from measured pilot volumes and current API pricing rather than presented here as an estimate, because a fabricated cost table would be exactly the kind of unverified claim this course warns against. The cost model and its live inputs are maintained alongside the evaluation harness.

6.5 Scaling Risks

Risk	Why it bites at scale	Control
Latency	Forty reports arriving near a 16:00 cut-off create a burst, and a slow queue at 18:00 is a useless queue	Asynchronous processing on the existing background worker and queue infrastructure, with results ready before the review window opens
Cost growth	Linear per-call cost is invisible at five campuses and material at forty	Pinned context size, batching, a monthly spend alarm, and capability-level cost attribution
Workflow complexity	Every campus wants its own exception, and exceptions accumulate into an unmaintainable rule set	Rules live as configuration data scoped by university rather than as branching code

Risk	Why it bites at scale	Control
Governance burden	More AI surfaces mean more prompts, more evaluation sets and more review overhead	A hard rule that a capability does not ship without an evaluation set and a named owner
Data-quality drift	Upstream spreadsheet habits change and silently degrade inputs	Deterministic validation before the model sees anything, plus drift monitoring on the weekly sample
Model or provider change	A provider updates a model and behaviour shifts without warning	Pinned model versions, the evaluation set re-run before adopting any new version, one-flag rollback to rules-only

6.6 Operations Implications

- Support model: the platform currently has one builder and owner. That is the most serious operational risk in this entire submission, and it is a people risk rather than a technical one. Documentation, a second engineer and a written runbook are prerequisites for Phase 2, not nice-to-haves.
- Release discipline: schema migrations, prompt versions and rubric versions are all versioned artifacts, and a release is not clean until the evaluation set has been run against the version being shipped.
- Change management: reviewers need to be told plainly what the layer does and does not do. The failure mode of an over-sold AI feature is not disappointment; it is misplaced trust.
- Training: fifteen minutes of onboarding per reviewer, focused almost entirely on when to distrust the suggestion.

7. Ethics, Governance and Risk Note

7.1 Main Risks

#	Risk	Severity
R1	Hallucinated or unsupported statements in a summary lead a reviewer to act on something that was never reported	High
R2	Automation complacency — reviewers stop reading the original reports and the oversight step becomes ceremonial	High
R3	A serious issue is ranked low, is signed off routinely, and surfaces days later as an escalation	High
R4	Campus or role scoping failure exposes one campus's operational data to another	Critical
R5	Personal data of students or staff is sent to a third-party model API without a clear basis	High
R6	Systematic bias against particular campuses, associates or writing styles — for example, associates writing in less fluent English being flagged more often	High
R7	Function creep — the layer is quietly repurposed for staff performance evaluation	High
R8	Over-claiming in internal communication creates expectations the system cannot meet	Medium
R9	Key-person dependency on a single builder and owner	High

7.2 Harm Scenarios

Naming who gets hurt, specifically, is what turns a risk register into a governance document.

- A Business Operations Associate honestly reports a genuine problem in plain, non-fluent English. The model rates it low because it lacks urgent phrasing. The problem persists for a week, students are affected, and the associate learns that reporting honestly achieved nothing — which quietly degrades the quality of every future report.
- A summary states that a fee issue was "resolved" when the report said it was "being resolved". The reviewer signs off, and a student is blocked from class the next day.
- Operational text that includes a named student's personal circumstances is sent to an external API without a considered basis, and the institution has no answer when asked where that data went.
- A manager begins using severity distributions as an informal ranking of associates. Associates learn to write reports that score well rather than reports that are true. The platform has then made the institution less informed while appearing more informed.

7.3 Controls

Control	Addresses	Implementation status
No autonomy — AI cannot advance any state or send any message	R1, R2, R3	Enforced by the existing state machine; the model writes only to its own suggestion table

Control	Addresses	Implementation status
Mandatory source citation for every generated claim, shown in the interface	R1	Designed for the pilot
Rules can raise severity but never lower it	R3	Designed for the pilot
Auto-sign-off disabled for any item marked serious	R2, R3	Configuration change to an existing platform behaviour
Two-sided override band monitored weekly with reasons read by a human	R2	Part of the evaluation harness
Row-level security enforcing campus scoping at the database, not only in application code	R4	Live in production today
Data minimisation — personal identifiers redacted from text before the API call wherever the task does not need them	R5	Designed for the pilot; a launch prerequisite
Bias audit on the weekly sample, comparing severity distributions across campuses and report authors	R6	Part of the evaluation harness
Written policy that AI outputs are never used in staff performance assessment, communicated to all users	R7	Governance commitment, to be signed off before the pilot
Immutable audit log capturing model output, prompt version and the human decision together	R1–R7	Live in production today
Documentation, runbook and a second engineer before Phase 2	R9	Open — explicitly a gate on expansion

7.4 Privacy Position

Operational text can incidentally contain personal information about students and staff. The position taken is data minimisation before capability: identifiers are stripped from text sent to the model wherever the task does not require them, because severity classification of an operational issue almost never depends on knowing which named student was involved. Where an identifier is genuinely needed, the reference stays inside the platform and the model receives a placeholder. Retention of model inputs and outputs follows the platform's existing retention policy, and the audit log records what was sent for every call. Alignment with the institution's data-protection obligations, including India's Digital Personal Data Protection Act, is treated as a prerequisite for the pilot rather than a follow-up action.

7.5 Launch Blockers

Any one of the following stops the launch. They are stated as absolutes deliberately: a blocker that can be negotiated under delivery pressure is not a blocker.

- Any cross-campus or cross-role data exposure, even once, even in a test.
- Ungrounded statements above five per cent of sampled summary statements.
- P1 severity recall below 0.90 on the labelled weekly sample.
- No data-minimisation step in place before text is sent to the model API.
- Absence of a working single-flag rollback to rules-only ordering.
- Any proposal to use the layer for staff performance evaluation.

8. Prototype and MVP

8.1 Prototype Type

The prototype is not a mock-up. It is a working, production web application that the campus network uses to run its day, with the Intelligence Layer implemented on top of it in two states: capabilities already shipped, and the pilot wedge built for evaluation. This is a meaningful advantage for a capstone, because the usual weakness of a prototype — that it proves nothing about real workflow fit — is partly answered here by real users doing real work. It also carries an obligation to be precise about which parts are live and which are designed, and Section 8.5 does exactly that.

8.2 Modules in the Platform

The Intelligence Layer is only credible because it sits on a platform that already runs the operational day. The modules below are live in production. Each one is deterministic, none of them hands a decision to a model, and each one produces the structured context that the triage layer is grounded in.

Module	What it does today
Fee Collection	Semester-window fee tracking synchronised from institutional sheets on a short cycle, twice-daily snapshot reporting, per-batch and per-coach rollups, dropout tracking
Daily Operations Submission Spine	One daily report per campus by the 16:00 cut-off and sign-off by 18:30, on a strict state machine (draft, submitted, in review, signed off, sent back, locked), with reminders, auto-sign-off and immutability once locked
Task Centre	Assignable tasks with multiple assignees, checklists and a view log, so it is clear who saw what and when
Budget Proposals and Petty Cash	Multi-stage approval workflow for campus spend, with a two-level approval path, event completion and post-event report submission closing the loop, and an approval webhook into the facilities system. Described in detail in 8.2.1
Email Campaigns and Mailboxes	Per-university mailboxes, mail-merge templates, bulk send with per-recipient open and acknowledgement tracking, and a full mail audit log
Assessments and Exam Papers	Question bank, per-university paper templates, generation of multiple distinct but equivalent paper sets, and import from document and spreadsheet formats
Faculty and Success Coach Tracking	Faculty profiles and subject mappings that drive scheduling, daily attendance, instructor logs and success-coach call targets
Communication Tasks	Multi-channel staff outreach with an escalation ladder and push notification
Meeting Intelligence	Discovers scheduled meetings, pulls the transcript, and produces a summary, action items and attendance
Schedule Events	Holidays, exams, events and curricular items, with checklists, procedure documents and calendar freezes
Academic Operations Hub	The per-university command centre: examinations, faculty operations, student operations and setup

Module	What it does today
Notifications	Unified in-app, email and push notification with deduplication and delivery tracking

8.2.1 Budget Proposals — the worked example of where AI is deliberately absent

Budget proposals deserve describing in detail, because they are the clearest illustration of the product judgement in this submission. A campus raises a proposal for spend: an event, a repair, a purchase. It moves through a fixed approval workflow — drafted at the campus, submitted for review, reviewed, approved at the first level, approved finally, and then, once the money is spent, marked as event completed, with a report of actuals submitted before the proposal is closed. Proposals above a defined value threshold require an additional administrative approval, and a final approval fires a webhook into the facilities system so that fulfilment starts without anyone re-typing anything. Petty cash is tracked against the same spine.

Every step of that is deterministic code. There is no model anywhere in the money path, and there will not be one. Approval authority, spend thresholds, state transitions and the audit record have to be exactly correct and defensible to an auditor, and a probabilistic system cannot offer that guarantee. This is the concrete answer to the question of where AI is not the right lever.

There is still a genuine AI opportunity beside that path rather than inside it, and it is scoped as P2 in the opportunity matrix. Proposals routinely arrive incomplete — a missing quote, a vague justification, a cost line that does not match the narrative — and the reviewer discovers this only after opening the proposal, which sends it back and adds days. A completeness check would read the proposal text and its attachments before submission, tell the submitter what is missing, and flag proposals that look like duplicates of spend already approved for the same campus in the same window. That is drafting help and pattern spotting, and both fail safely: the worst case is a wrong suggestion shown to a person who is about to press submit anyway. Pending budget approvals also appear as one of the numeric signals in the daily triage context, so a campus sitting on a stalled approval becomes visible in the evening review instead of surfacing as a complaint a week later.

> **THE LINE THAT DOES NOT MOVE**
>
> The rule the platform follows: AI reads, drafts and ranks. Money, permissions, approvals and audit records stay deterministic. Every module listed above respects that line.

8.3 System Shape

Layer	Implementation
Application	SvelteKit web application, role-scoped page routes, deployed on Railway
Data	Supabase PostgreSQL, event-sourced, with row-level security enforcing university and role scoping in the database
Background processing	An independent worker deployment running scheduled loops (fee synchronisation, snapshots, reminders, auto-sign-off, scheduled AI reports) plus job queues for notifications and meeting processing
Caching and queues	Redis with a job-queue library for asynchronous work
AI services	Google Gemini for summarisation, extraction and narrative reporting; Google Vision for OCR

Layer	Implementation
Integrations	Google OAuth for sign-in, Gmail, Calendar, Sheets and Drive; push notifications; a webhook into the facilities system
Core model	The university is the scoping key on nearly every table, and a role-to-permission matrix gates every page and endpoint — adding a university makes it a filter everywhere, and changing a role changes what exists for that user

8.4 What the Prototype Demonstrates

- The full daily decision loop: submission, deadline, AI-assisted triage, human review, sign-off or escalation, immutable audit.
- The confidence-routing behaviour — that a low-confidence item genuinely looks and behaves differently from a high-confidence one, rather than merely displaying a different label.
- The separation between deterministic facts and generated text, visible in the interface rather than only in the architecture.
- The override path with reason capture, and the weekly evaluation view built from those overrides.
- That the workflow degrades safely: with AI switched off, the product is still the platform the network already runs on.

8.5 What It Does Not Yet Prove

Stating this plainly is part of the product argument, not an apology for it.

- It does not prove sustained severity accuracy. Four weeks of labelled samples is enough for a go/no-go decision and not enough for a claim about the year.
- It does not prove that reviewers keep reading. Automation complacency is a behaviour that develops over months, and one month of pilot cannot rule it out.
- It does not prove the cost ceiling holds at forty campuses, since the pilot measures five.
- It does not prove absence of bias across campuses and writing styles; it establishes the measurement that would detect it.
- It does not prove supportability. One builder is not an operating model, and no demonstration changes that.
- Examination classroom allocation has not yet been exercised across a full examination cycle at every campus, so it is evidence of a working method rather than of network-scale reliability.

8.6 Links and Evidence

Artifact	Location
Live application	https://uniconnect-app.up.railway.app — sign-in is Google OAuth and access is role-scoped, so an evaluator account is provisioned on request
Repository	https://github.com/programopscentral-art/Uniconnect-mail-automation- — public, SvelteKit and TypeScript monorepo with the worker, shared packages and migrations
Screenshots of live modules	Appendix — Prototype Evidence: all 66 captured screens, captioned and grouped (separate document)

Artifact	Location
Pilot wireframes for the triage queue	Appendix/Wireframes/
Evaluation harness and rubric	Appendix/Metrics/
Walkthrough video (5 minutes)	Appendix/Screenshots_or_Video/
Setup instructions	04_Prototype_Readme_or_Link.md

Because the system carries live institutional operational data, public access cannot be granted. The walkthrough video and screenshots use a demonstration campus with representative but non-identifying data.

9. Examination Classroom Allocation — Optimisation AI in the Examinations Module

9.1 Why This Belongs in the Submission

Allocating students to examination classrooms is the second AI capability in the platform, and it is a different species from the triage layer. There is no language model anywhere in it. It is a constrained combinatorial problem — seat every candidate, respect every room capacity, and satisfy an adjacency rule that cannot be broken — solved by construction, checking and repair. It is included because the course explicitly asks teams to distinguish AI as a lever from AI as a fashion, and this is the clearest case in the platform where the right technique is classical optimisation rather than generation.

> **THE CENTRAL INSIGHT**
>
> The two AI capabilities in this platform fail in opposite ways. The language layer fails loudly and plausibly — it says something confident and wrong, and a careful reader can catch it. The allocation layer fails quietly and correctly-looking — it produces a complete, tidy seating chart with one adjacency violation buried in row nine that nobody notices until the exam is being written. That is why this section is mostly about invariants and flags rather than about the algorithm.

One scope point, stated plainly because it is easy for a reader to assume otherwise: regular class timetabling is not part of this platform. UniConnect does not generate weekly or semester class schedules. What it does is examination-side allocation — which candidates sit in which room, in which seat, writing which paper set, with which invigilator. Everything in this section refers to that, and nothing in it should be read as a claim about class timetabling.

9.2 The Workflow Being Replaced

Before this existed, seating for each examination was built by hand. A Business Operations Associate took the candidate list, the available rooms and the number of paper sets, and produced a seating chart in a spreadsheet, usually the evening before. Three things went wrong repeatedly. Rooms were filled unevenly, so one hall was crowded while another sat half empty. Adjacency was checked by eye, which is exactly the kind of check human attention fails at across a few hundred seats. And when anything moved — a room became unavailable, candidate numbers changed after a late registration — the whole chart was rebuilt from the start rather than adjusted.

The cost was not only the hours. A seating chart that is wrong is discovered in the room, in front of students, minutes before an examination begins, and the only remedies available at that point are bad ones.

9.3 The Problem, Stated Precisely

Element	Detail
Inputs	The candidate list for the examination, the rooms available with their capacity and seat layout, the examination slot, and the number of distinct paper sets generated for that paper
Hard constraint	No two candidates seated side by side may come from the same branch or section — the interleaving constraint. This is an invariant, not a preference: a plan that breaks it is not a worse plan, it is an invalid one. The generator reports a per-classroom violation count, and the seat map carries an explicit constraint-violation state

Element	Detail
Hard constraint	Room capacity is never exceeded, and every candidate is seated exactly once
Objective	Fill rooms evenly rather than packing the first room and thinning the last, and keep the allocation traceable so a candidate can be found by roll number in seconds
Allocation modes	Section-level, where a whole section receives one set and sets cycle across sections, or seat-level, where the set alternates seat by seat within a room
Simplifying option	Co-locating cohorts from different branches in the same room makes interleaving easier to satisfy, because adjacent candidates are already writing different subjects
Downstream output	The approved allocation drives the invigilation assignment and the printed room and seat lists

9.4 How the Allocation Works

1. Feasibility is checked before anything is placed: enough seats across the available rooms, and enough distinct paper sets for the chosen allocation mode. If the inputs cannot produce a valid plan, the system says so and names the shortfall rather than producing a plan that quietly bends a rule.
2. Candidates are placed by a round-robin fill across rooms, which is what produces even occupancy instead of a full first hall and an empty last one.
3. Paper sets are assigned according to the chosen mode — one set per section cycling across sections, or alternating seat by seat within the room.
4. A repair pass then walks the completed layout and fixes any position where the interleaving invariant is violated, swapping seats rather than restarting the allocation. Each classroom reports its violation count, and a clean plan reads zero.
5. If a valid plan cannot be reached — most often because one branch is so dominant in the room that interleaving cannot be satisfied — the plan is flagged as unsafe and shown to the operator with the reason. It is never silently emitted as though it were valid.

> **THE LINE THAT DOES NOT MOVE**
>
> The design rule this follows: an allocation that cannot satisfy the invariant must fail visibly. A seating chart that looks finished is trusted completely by the people using it, so an unsafe plan that is presented normally is worse than no plan at all.

9.5 Invigilation Assignment

Invigilator assignment sits immediately downstream of the seating plan and follows a conflict-of-interest ladder rather than a convenience order. Instructors teaching other subjects are preferred first, chosen by who is least loaded so far in the examination period; Business Operations Associates come next; instructors of the subject being examined are used only when nothing else is available, and that case is surfaced as an alert rather than accepted silently. Load is balanced across the whole examination period rather than per day, because per-day balance produces a fair-looking schedule with the same few people carrying the fortnight.

9.6 Disruption and Change

Examinations move. A room becomes unavailable, a date shifts, a cohort is added. A change of that kind is handled as one cascade rather than as a manual rebuild: the affected examinations are

identified, the approved change is recorded as an audit entry, the allocation is recomputed for the affected rooms, invigilation is recalculated downstream, and everyone affected is notified from the same event. The candidate-facing output — room and seat lists — is regenerated from the recomputed plan rather than edited by hand, so the printed sheet and the system never disagree.

9.7 Human in the Loop

- A generated allocation is a draft. It becomes the operative plan only when a person approves it, and the approval is recorded.
- Unsafe or flagged plans cannot be approved without an explicit acknowledgement of the specific reason they were flagged.
- Manual seat adjustments are permitted, and the invariant is re-checked after any manual change — the human can override the plan, but not the check.
- Every allocation, approval and change is written to the same immutable audit trail as the rest of the platform, which matters because examination records are exactly the kind of record that gets questioned months later.

9.8 Status, Evidence and What This Does Not Prove

- The allocation logic and its invariant are implemented within the Examinations module of the Academic Operations Hub. It has not yet been exercised across a full examination cycle at every campus, so what exists is evidence of a working method rather than of network-scale reliability.
- The measures that matter are simple and countable: adjacency violations in an approved plan, which must be zero; the spread of occupancy across rooms; the time from candidate list to approved seating plan; and the number of plans that had to be repaired or flagged, which is the early-warning signal that the inputs, not the algorithm, are the problem.
- Examination paper generation and exam scheduling are designed as part of the same module and are at different stages of build. Nothing in this section should be read as a claim that the whole examination lifecycle is complete.
- Regular class timetabling remains out of scope for this platform, as stated in 9.1.

> **RUN IT, DO NOT READ IT**
>
> One working practice is worth recording, because it changed outcomes more than any design decision did. Reading the code and reasoning about what a system should do has produced almost every wrong conclusion in this project; running it against real data has produced almost every real finding. A check that runs and passes can still be checking the wrong thing. For a seating plan, that means the honest test is not "does validation pass" but "has anyone actually walked row nine of the generated chart".

10. Course Case Reference and Lessons Transferred

10.1 Primary Lens — Swiggy and Invisible AI

The Swiggy case is the closest match to this product, and the resemblance is structural rather than superficial. In both, AI sits inside an operational marketplace of people doing physical work, and the user never sees a model — they see an estimate, an assignment or an ordering. The intelligence is invisible, embedded in a workflow that would exist without it.

Question	Answer for this project
What is similar?	AI is embedded in an operational loop rather than exposed as a conversational surface. Value comes from a small decision made many times a day, and the system must degrade gracefully because the operation cannot stop when a model does.
What is different?	Swiggy's AI decisions are largely reversible within minutes and diffuse across millions of orders. Ours attach to a named campus and a named associate, which raises the cost of an individual error and makes explainability a requirement rather than a nicety. Our volume is also thousands of decisions per month, not millions, so we cannot rely on scale to average out mistakes.
What lesson transfers?	Make the AI invisible and the workflow visible. Users should experience a better-ordered evening, not an AI feature. Reliability and the fallback path matter more than model sophistication.
What would we do differently?	We deliberately keep a mandatory human decision in the loop where Swiggy automates, because our error cost is concentrated rather than diffuse. We also expose evidence for every suggestion, which a consumer marketplace has no need to do.

10.2 Supporting Lenses

Case	Lesson taken	How it shows up in this submission
Netflix — metrics and evaluation	A model metric that is not tied to an outcome metric will be optimised into irrelevance	Every technical metric in Section 5.6 is paired with the business outcome it is meant to move, and guardrail metrics can veto a launch
iMerit — data operations and labelling	Labelling quality and rubric clarity beat labelling volume	A 30-report weekly labelled sample with a versioned rubric, where labeller disagreement is treated as a rubric defect
Bias in lending / Apple Card lens	Bias appears through proxies nobody designed in	Writing fluency is our proxy risk, so the bias audit compares severity distributions across campuses and report authors
Instacart and Tesla — lifecycle and drift	Systems degrade quietly as the world changes around them	Pinned model versions, evaluation re-run before adopting a new version, and drift monitoring on the weekly sample
OpenAI scaling and pricing	Per-call cost is invisible in a pilot and decisive at scale	Pinned context size, spend alarms, cost attribution per capability, and a unit-economics test stated per campus-month

11. AI Collaboration Log

This section discloses how AI tools were used to produce this submission and the product artifacts behind it. It is written to be honest rather than flattering, because a collaboration log that reports no failures is not a log.

11.1 Tools Used

Tool	Used for
Claude	Structuring this document, drafting and rewriting PM artifacts, generating edge cases, stress-testing the risk register and metric design, and reviewing prototype code
Google Gemini	Inside the product itself — daily and weekly narrative summaries, meeting summarisation and document analysis
AI coding assistance	Scaffolding components, migrations and repetitive interface code during platform development
Google Vision	Inside the product — OCR for document intake

11.2 Where AI Helped

- Discovery: turning scattered field notes from campus visits into a structured problem list, which made repeated patterns visible faster than reading the notes again would have.
- Drafting: producing first versions of PRD sections and this report's structure, which removed the blank-page cost and let effort go into judgement.
- Edge cases: generating long candidate lists of failure scenarios across scheduling, seating, invigilation and paper generation. This is the single highest-value use — the model reliably proposes cases a tired human forgets.
- Coding: scaffolding repetitive parts of a large application quickly.
- Metrics: proposing alternative metric formulations, including the two-sided override band, which came out of pushing back on a first draft that had only a lower bound.
- Risk review: challenging the risk register and surfacing the function-creep scenario that became R7.

11.3 Example Prompts and Workflows

#	Prompt or workflow	What came back and what was done with it
1	Given this daily operations workflow and this database schema, list every way an AI triage layer could fail in a way the reviewer would not notice.	A long list, roughly two-thirds useful. It produced the automation-complacency framing that became R2 and reshaped the override metric. Generic entries about "model bias" were discarded as unactionable.
2	Argue against building this feature. Assume a competent manager who thinks a rules engine would be enough.	The strongest single prompt used. It forced the rules-only baseline in Section 5.4, which is now a running comparison in the pilot rather than a rhetorical claim.
3	Draft severity classification rubric options for operational campus reports, with worked examples per level.	Three rubric drafts. All were too abstract initially; usable only after real report examples were supplied, and then rewritten by hand.

#	Prompt or workflow	What came back and what was done with it
4	Review this PRD section and identify claims that are asserted without evidence.	Accurately caught several unsupported efficiency claims, which were either removed or moved into the assumptions section.
5	Propose an exam timetable generation approach for cohorts with shared subjects across sections.	Suggested graph-colouring with a conflict graph, which was a genuinely useful starting point, but the initial version ignored the requirement to maximise the gap between a student's consecutive exams — added by hand afterwards.

11.4 Where AI Was Weak or Wrong

- Generic personas and workflows. Every unprompted persona was a stock "operations manager" with no resemblance to how a Business Operations Associate actually spends an afternoon. Only field notes fixed this.
- Unrealistic automation ambition. Default suggestions consistently pushed towards full automation and end-to-end agents, with no weight given to the cost of a wrong decision in an academic setting.
- A live grounding failure in the product itself. On the day these screens were captured, the shipped Ops Copilot summary stated that there was a complete absence of operational activity and zero values across all indicators, on the same screen that reported 355 of 369 sessions completed and 30.6 per cent attendance. It is reproduced in the appendix. This is the clearest evidence in the submission for risk R1, and it is the reason the layer is assist-only: a reviewer who trusted that paragraph instead of the figures beside it would have drawn exactly the wrong conclusion about a campus.
- Invented numbers. When asked for efficiency estimates, plausible-looking percentages were produced with no basis. These were deleted; unmeasured figures in this document are labelled as assumptions rather than replaced with confident-sounding ones.
- Weak prioritisation. Ranking suggestions were sensitive to the order in which features were listed and did not weigh the cost of error until that dimension was explicitly added.
- Shallow governance framing. Risk sections came back as compliance boilerplate. The specific harm scenarios in Section 7.2 came from knowing the people who write these reports, not from a model.
- Over-confident architecture. Suggested designs occasionally ignored constraints already stated, such as the requirement that permission scoping be enforced at the database layer rather than in application code.

11.5 Human Corrections Applied

AI proposal	Human correction	Reasoning
Fully automated report approval for high-confidence items	Assist-only; no automatic state advancement of any kind	The error cost is concentrated on a named campus, and the oversight step is the entire safety argument
Predictive student dropout scoring in the MVP	Deferred out of the MVP entirely	Training data reflects recording habits more than student behaviour, and a wrong prediction attaches to a real student

AI proposal	Human correction	Reasoning
Override rate with a lower bound only	Two-sided band, 10% to 25%	A very low override rate is a warning sign of rubber-stamping, not a success signal
Broad multi-module AI rollout	One wedge, five campuses, one month	Only a narrow scope produces an evaluation anyone should believe
Estimated efficiency gains stated as facts	Rewritten as pilot targets and assumptions	Fabricated evidence would invalidate the entire argument
Model-generated numeric analysis	All numbers stay deterministic; the model never computes	Arithmetic must be exactly correct and is trivially handled in code

11.6 Final Reflection

AI accelerated the parts of product management that are production work: structuring documents, drafting, enumerating edge cases, generating alternatives and scaffolding code. On edge-case generation in particular it was better than working alone, and the exam-operations design is more complete because of it.

It could not do the parts that are actually the job. It did not know which problem was worth solving, because that came from standing in campuses and watching people work. It did not know what error cost the institution could tolerate. It could not decide that automated staff scoring should be rejected outright rather than deferred, and when asked, it produced a balanced discussion instead of a decision. It could not tell the difference between a number that was measured and a number that sounded right — which is, in the end, the difference between a product recommendation and a pitch.

> **THE PRACTICAL LESSON**
>
> The most useful pattern found was adversarial rather than generative: asking the model to argue against the product, to find unsupported claims, and to list ways the system could fail invisibly. Used that way it is a demanding reviewer. Used as an author, it produces confident, generic, unfalsifiable work.

12. Individual Contribution Disclosure

This is an individual submission. All artifacts were produced by the submitter, with AI assistance disclosed in Section 11.

Artifact	Led by	Nature of the work
Problem identification and field research	Manoj Kumar Panchothi	Campus visits, operator conversations and daily use of the system
Product strategy and opportunity framing	Manoj Kumar Panchothi	Written independently; opportunity matrix scored and defended personally
PM artifact pack (PRD, prioritisation, MVP, roadmap, metrics)	Manoj Kumar Panchothi	Drafted with AI assistance, restructured and decided by hand
UX, workflow and trust design	Manoj Kumar Panchothi	Screen and interaction design, confidence-band behaviour
Data, model and evaluation strategy	Manoj Kumar Panchothi	Task decomposition, rubric design, evaluation harness and thresholds
Business, economics and scaling view	Manoj Kumar Panchothi	Cost model structure and unit-economics test
Ethics, governance and risk	Manoj Kumar Panchothi	Risk register, harm scenarios, controls and launch blockers
Prototype and platform build	Manoj Kumar Panchothi	Sole builder and owner of the production platform
AI collaboration log and this report	Manoj Kumar Panchothi	Written and verified personally

12.1 How Outputs Were Validated

- Product claims were checked against the live system rather than against memory, and anything that could not be checked was marked as an assumption.
- Workflow descriptions were validated against how operators actually work, not against how the system was designed to be used.
- AI-generated content was treated as a draft in every case, and no section of this document is unedited model output.

13. Assumptions and Evidence Quality

This section exists so that a reader can tell, at a glance, which parts of the argument rest on measurement and which rest on judgement. It is the section a leadership review group would ask for first.

Claim or input	Evidence quality	How it becomes stronger
The platform is in production across the campus network and is the daily system of record	Measured — live system, live users	Already firm
Campus scoping is enforced at the database layer	Measured — implemented and testable	Already firm; verified by targeted tests before the pilot
Gemini summarisation and meeting intelligence are shipped and in use	Measured — running today	Already firm
Reviewers cannot read 40 reports carefully in the evening window	Qualitative — operator conversations and direct observation	Baseline timing study during the first pilot week
Serious issues currently surface one to two days late	Qualitative — recalled incidents, not yet quantified	Measure the write-to-escalate interval from the audit log for the pre-pilot period
A 40% reduction in reviewer time per report is achievable	Assumption — a pilot target, not a finding	Measured directly against the pre-pilot baseline
Inference cost stays within the modelled ceiling at 40 campuses	Assumption — modelled from pilot volumes at 5 campuses	Extrapolate from measured pilot volumes; re-test at 20 campuses
Reviewers will continue reading source reports under AI assistance	Unknown — this is the central open risk	Two-sided override band plus structured reviewer feedback; still only partially answerable in one month
Severity classification can reach 0.90 recall on serious issues	Unknown — the primary hypothesis under test	The weekly labelled sample is the test

13.1 Limitations of This Submission

- The pilot has not run for a full academic month at the time of writing, so the go/no-go thresholds are stated but not yet resolved.
- The submitter is both the builder and the evaluator of the platform, which is a real source of bias. The weekly labelled sample is deliberately produced by a reviewer who is not the submitter, partly for this reason.
- Operational data cannot be shared publicly, so the evidence presented to the evaluator is screenshots, a walkthrough and access on request rather than an open dataset.

Appendix A. Topic Approval Form

Field	Entry
Team name	UniConnect (individual submission)
Team members	Manoj Kumar Panchothi
Working title	UniConnect Operations OS — AI-Assisted Academic Operations for a 40-Campus Network
Industry / function	Education operations — multi-campus academic programme management
Target user	Programme Managers and the Chief of Staff, COS (reviewers); Business Operations Associates, BOAs, and Programme Management Associates (submitters); central academic operations
Current workflow problem	Around 40 free-text daily campus reports plus attached numbers must be reviewed in a short evening window. Serious issues are buried in routine text, surface late, and the reasoning behind escalation decisions is not recorded.
Why this matters	Operational delay in an education network is paid for in student experience — blocked class access, unresolved facility problems, exam-day failures — and the cost of reading grows linearly with campus count.
Proposed AI role	Extraction, classification, grounded summarisation and confidence-based routing — an assistive triage layer, not an autonomous decision-maker
Why AI instead of a simpler alternative	Numeric thresholds and dashboards already exist and already catch numeric breaches. They cannot read a complaint expressed in prose by forty different writers. A rules-only baseline runs alongside the pilot precisely so this claim is tested rather than assumed.
Human-in-the-loop design	The existing daily sign-off state machine is the control point. No AI output advances a state or sends a message. Overrides require a structured reason. Auto-sign-off is disabled for items marked serious.
Smallest credible MVP	Daily operations triage on five campuses in one cluster for one academic month, assist-only, with a weekly labelled evaluation sample and a rules-only comparison running in parallel.
Build vs buy vs API view	API for models and OCR; build the workflow, permissions and audit; buy managed infrastructure; partner with the existing Google Workspace estate.
Main risks	R1 hallucinated summaries acted upon; R2 automation complacency turning oversight into ceremony; R4 campus-scoping failure exposing one campus's data to another
Success evidence	P1 severity recall at or above 0.90 on the labelled sample; reviewer time per report down by at least 40%; override rate inside the 10–25% band; zero scoping violations; reviewers reporting that the queue changed their working order for the better
Team recommendation	Proceed — pilot on five campuses, with expansion gated on the thresholds in Section 5.7

Appendix B. Requirement Coverage

Requirement	Coverage	Where it is evidenced
The problem is specific	Met	Sections 1.1 and 3.1 — one named decision, one named review window
The AI role is clear	Met	Sections 1.4, 3.3 and 5.1 — AI reads, classifies and drafts; it never computes, approves or sends
The product strategy is defensible	Met	Section 2 — layered build/buy/API decision with stated trade-offs
The PM artifacts are complete	Met	Section 3 — problem, vision, PRD-lite, prioritisation, MVP, roadmap, metrics
The UX handles uncertainty	Met	Section 4.5 — four confidence bands with distinct system behaviour
The evaluation logic is meaningful	Met	Sections 5.3 to 5.7 — real ground truth, two baselines, thresholds set in advance
The business and scaling note is realistic	Met	Section 6 — unit-economics test, cost drivers, scaling risks, key-person risk named
The ethics and risk note is specific	Met	Section 7 — nine risks, named harm scenarios, control-to-risk mapping, absolute blockers
The prototype shows the core value	Met	Section 8 — production system plus pilot wedge, with limits stated in 8.4
The AI collaboration log is honest	Met	Section 11 — includes where AI was wrong and what was deleted
The recommendation is clear	Met	Section 1.5 — pilot, with the conditions for scaling stated

End of document.